

Towards Open-World Retrieval-Augmented Generation on Knowledge Graph: A Multi-Agent Collaboration Framework

Jiasheng Xu*
School of Computer Science and
Cyber Engineering,
Guangzhou University,
Guangzhou, Guangdong, China
Institute of Automation, Chinese
Academy of Sciences,
Beijing, China
jiasheng@e.gzhu.edu.cn

Mingda Li*
Institute of Automation, Chinese
Academy of Sciences,
Beijing, China
limingda2018@ia.ac.cn

Yongqiang Tang†
Institute of Automation, Chinese
Academy of Sciences,
Beijing, China
yongqiang.tang@ia.ac.cn

Peijie Wang
Institute of Automation, Chinese
Academy of Sciences,
Beijing, China
wangpeijie2023@ia.ac.cn

Wensheng Zhang
School of Computer Science and
Cyber Engineering,
Guangzhou University,
Guangzhou, Guangdong, China
Institute of Automation, Chinese
Academy of Sciences,
Beijing, China
zhangwenshengia@hotmail.com

Abstract

Large Language Models (LLMs) have demonstrated strong capabilities in web search and reasoning. However, their dependence on static training corpora makes them prone to factual errors and knowledge gaps. Retrieval-Augmented Generation (RAG) addresses this limitation by incorporating external knowledge sources, especially structured Knowledge Graphs (KGs), which provide explicit semantics and efficient retrieval. Existing KG-based RAG approaches, however, generally assume that anchor entities are accessible to initiate graph traversal, which limits their robustness in open-world settings where accurate linking between the user query and the KG entity is unreliable. To overcome this limitation, we propose **AnchorRAG**, a novel multi-agent collaboration framework for open-world RAG without the predefined anchor entities. Specifically, a predictor agent dynamically identifies candidate anchor entities by aligning user query terms with KG nodes and initializes independent retriever agents to conduct parallel multi-hop explorations from each candidate. Then a supervisor agent formulates the iterative retrieval strategy for these retriever agents and synthesizes the resulting knowledge paths to generate the final answer. This multi-agent collaboration framework improves retrieval robustness and mitigates the impact of ambiguous or erroneous

anchors. Extensive experiments on four public benchmarks demonstrate that AnchorRAG significantly outperforms existing baselines and establishes new state-of-the-art results on the real-world reasoning tasks. The datasets along with our code are available at <https://github.com/Jayson3831/AnchorRAG>.

CCS Concepts

• Information systems → Language models.

Keywords

Retrieval-Augmented Generation; Knowledge Graph; Large Language Model; Agentic search

ACM Reference Format:

Jiasheng Xu, Mingda Li, Yongqiang Tang, Peijie Wang, and Wensheng Zhang. 2026. Towards Open-World Retrieval-Augmented Generation on Knowledge Graph: A Multi-Agent Collaboration Framework. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792389>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18310915>.

1 Introduction

Large Language Models (LLMs) [1, 42] are typically defined as deep learning models with a massive number of parameters, trained on large-scale corpora in a self-supervised manner. Their internal parameterization allows for an implicit representation of external knowledge. During the inference stage, the chain-of-Thought (CoT) [38] method guides the models to “think step-by-step” through

*Equal contribution.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792389>

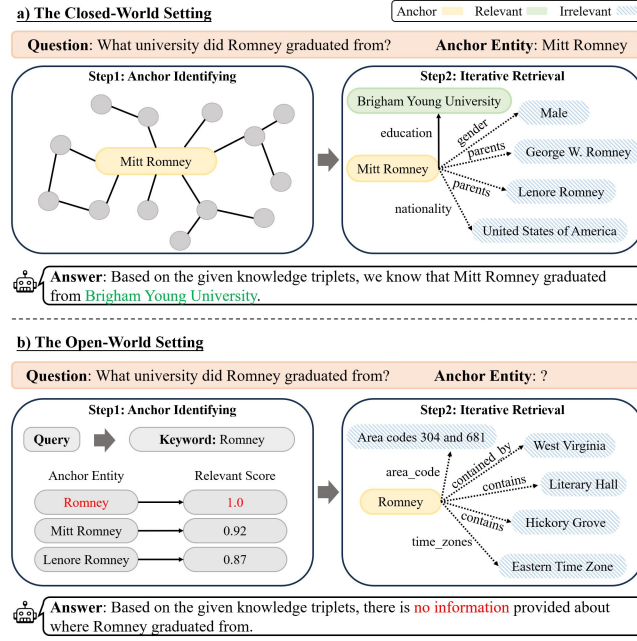


Figure 1: Illustration of the difference between the closed-world and the open-world setting in RAG.

carefully designed prompts. It helps the models better handle logically complex or multi-step reasoning tasks, achieving a better performance in natural language processing tasks such as web search [24]. Despite these advances, LLMs are fundamentally limited by the incompleteness and inaccuracies in their static training corpora. This often results in factual hallucinations [16], especially in knowledge-intensive tasks, significantly hindering their reliability and deployment in real-world applications.

To address these limitations, recent studies [17, 39] have investigated augmenting LLMs with external knowledge sources to improve factual consistency and reasoning. While fine-tuning [14] LLM with the external knowledge is computationally expensive and inflexible, a promising solution is Retrieval-Augmented Generation (RAG) [22], which combines real-time retrieval from external sources with generative modeling. To improve the retrieval efficiency of the external sources, some researchers apply the knowledge graph (KG) to represent the external information as the structured triples, i.e., (entity, relation, entity). Then the core issue of RAG on the knowledge graph is to locate the relevant triples for enhancing the reasoning capability.

Among existing solutions, iterative retrieval methods [6, 27, 31, 35, 41] has become a research hotspot in the field, due to the fact that they can handle the complex user queries by dynamically constructing and refining explainable reasoning paths through multi-round “retrieval-generation”. Typically, these RAG methods can be divided in two steps: (1) identifying a query-aware anchor entity to initiate retrieval; (2) performing iterative retrieval to construct the reasoning paths. Through the above steps, these methods can effectively exploit the external knowledge related to the query and improve the performance in the downstream reasoning task.

However, existing KG-based RAG methods almost follow the closed-world setting where the anchor entity is accessible and exists in the given KG. As shown in the Figure 1 (a), the anchor entity “Mitt Romney” is pre-defined for the question “What university did Romney graduated from?”. The RAG methods can directly locate the target entity for the following retrieval. Actually, user questions usually conform to the open-world setting where the anchor entity is unavailable. In that case, existing methods usually borrow from entity linking models and extract the keywords in the query to identify the candidate anchor entity by semantic matching. Due to name abbreviations and aliases, these methods usually suffer from the issue caused by imprecise or partial matching as shown in Figure 1(b). Then the challenge of the open-world RAG lies in how to accurately identify anchor entities to retrieve the relevant information.

To tackle this issue, we propose **AnchorRAG**, a multi-agent collaborative (MAC) framework that can perform RAG without the predefined anchor entities, thereby enabling more effective knowledge retrieval to enhance the LLM reasoning performance. Specifically, our framework follows a pipeline-based MAC framework, and a predictor agent first extracts the keywords from the given question and applies a semantic match model to obtain the candidate entities by their name description and structure neighborhood. Then, multiple retriever agents initiate parallel graph traversal with each candidate, conducting iterative retrieval to obtain the pivotal knowledge for the following reasoning. Finally, a supervisor agent will synthesize the retrieved evidence and determine whether an answer can be generated and formulate the retrieval process. The main contributions of this paper are summarized as follows:

- **General Aspect.** We emphasize the importance of identifying the accurate anchor entity for the open-world RAG. By integrating multi-agent collaboration into the workflow of question answering, we can enhance the generality of RAG methods and improve the performance in the open-world setting.
- **Methodologies.** For anchor identifying, we design an entity grounding strategy to locate the candidate anchors by entity name description and structural neighborhood. For knowledge retrieval, we propose a novel retrieval method with candidate ranking and evidence validation, which can capture the resultful knowledge paths to boost the LLM reasoning.
- **Experimental Findings.** Extensive experiments demonstrate that AnchorRAG consistently outperforms existing baselines on four public QA datasets, establishing state-of-the-art results on the real-world reasoning tasks.

2 Related Work

Knowledge graphs[2, 4] provide essential supplementation to the static knowledge of LLMs[9], effectively alleviating factual inaccuracies and hallucination issues during complex reasoning[45] tasks. Consequently, the efficient retrieval of factual information from KGs for LLMs has become a critical topic of research, which can be divided into three main groups: *Semantic Parsing*, *Single-Pass Retrieval* and *Iterative Retrieval* methods.

2.1 Semantic Parsing Methods

Semantic parsing methods aim to convert natural language questions into executable logical forms[21], such as S-expressions or SPARQL queries. The results from these queries are then used as external knowledge to enhance the LLM’s reasoning capabilities. Researchers have explored various strategies for generating and evaluating these logical forms. FlexKBQA[25] first samples a multitude of S-expressions and employs a LLM to convert them into natural language questions, creating program-question pairs which are then used for training lightweight models. Another typical method RNG-KBQA[44] resorts to a ranking model to select the most relevant logical forms based on a set of candidates. Additionally, ChatKBQA[26] introduces a “Generate-then-Retrieve” framework, which first uses a fine-tuned LLM to generate a logical form skeleton and then populates it with specific details via an unsupervised retrieval module. It is important to note that if the generated logical forms are inaccurate, the entire reasoning pipeline will be confused, leading to incorrect final answers. This may limit the reasoning ability of semantic parsing methods in real-world scenarios, where accurate logical forms are hardly to generate.

2.2 Single-Pass Retrieval Methods

Single-Pass retrieval methods enable the acquisition of all pertinent information, including entities, relations, or subgraphs, from a knowledge graph through a single retrieval operation[12, 28]. These methods can effectively reduce the computational costs and reasoning latency. HippoRAG [11] attempts to reduce the complexity and resources of retrieval process by constructing a schema-less KG, executing single-step retrieval guided by a personalized PageRank algorithm. Similarly, G-Retriever[13] formulates subgraph retrieval as a PCST optimization problem, creating a synergistic effect between the efficient retrieval and semantic prompting. Along this line, KnowGPT[47] leverages reinforcement learning to extract entire reasoning chains in a single pass. Furthermore, KG-GPT[20] generates a complete evidence graph in a single instance through sentence segmentation and graph retrieval. To sum up, through above methods simplify reasoning workflows and resources, they usually face inherent limitations in tackling complex multi-hop reasoning, where an initial retrieval that fails to identify critical evidence or filter the meaningless information may easily lead to generate incorrect answers.

2.3 Iterative Retrieval Methods

Iterative retrieval frameworks are designed to tackle complex queries by dynamically constructing and refining reasoning paths through multi-round “retrieval-generation” operation[12, 46]. Early pioneering work, including ToG[31] and StructGPT’s IRR framework[18], demonstrates the viability of LLMs to progressively navigate KGs and accumulate evidence. To further handle the issues caused in the highly complex reasoning, recent research resorts to more sophisticated strategies, such as multi-stage optimization in GRAG[15], dynamic knowledge completion in Generate-on-Graph[41], multi-agent deliberation in Generate-on-Graph[27], and adaptive planning and backtracking mechanisms in Plan-on-Graph[6]. Despite their success, these advanced methods are fundamentally built on

an idealized assumption: that entities within a query can be easily linked to their corresponding nodes in the knowledge graph. To extend to the open-world settings, some methods explore to apply entity linking strategy [23, 30, 34] to identify these entities. For instance, ToG[31] and Paths-over-Graph[33] prompt LLM to automatically extract relevant entities, while AMAR[40] uses the ELQ[23] model, which leverages a dual-encoder architecture to simultaneously detect mentions and link them to corresponding entities. However, these methods usually suffer from the issue caused by imprecise or partial matching, especially when the queries involving abbreviations, aliases, or long-tail entities.

3 Preliminary

Knowledge Graph is commonly defined as a directed relational graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$ denotes the set of entities, $\mathcal{R}$ denotes the set of relations, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the set of factual triples. Each triple $(h, r, t) \in \mathcal{T}$ represents a factual statement, where $h \in \mathcal{E}$ is the head entity, $r \in \mathcal{R}$ is the relation, and $t \in \mathcal{E}$ is the tail entity. For example, a triple $(Barack\ Obama, born_in, Hawaii)$ encodes the fact “Barack Obama was born in Hawaii.”

Reasoning Path is defined as a sequence of linked triples in $\mathcal{G}$ that connect a source entity e_s to a target entity e_t . Formally, a path of length L can be written as:

$$\mathcal{P} = \{(e_0, r_1, e_1), (e_1, r_2, e_2), \dots, (e_{l-1}, r_l, e_l)\}, \quad (1)$$

where $e_0 = e_s$ and $e_l = e_t$. Reasoning paths capture multi-hop dependencies that are crucial for multi-hop question answering.

Reasoning Subgraph is a connected subgraph $\mathcal{G}_q \subseteq \mathcal{G}$ constructed during the answering process for a query q . Typically, $\mathcal{G}_q = (\mathcal{E}_q, \mathcal{R}_q, \mathcal{T}_q)$ is iteratively expanded by retrieving the k -hop neighborhoods of frontier entities and refined to retain only the query-relevant triples. Such subgraphs serve as the evidence basis for reasoning.

Knowledge Graph Question Answering (KGQA) is the task of mapping a natural language query q into one or multiple reasoning paths $\mathcal{P}$ or a reasoning subgraph $\mathcal{G}_q$, ultimately leading to the correct answer entity (or entities). The task requires both semantic understanding of the query and logical navigation in the KG.

Open-World KG-RAG extends the KGQA by relaxing the assumption that a set of ground-truth anchor entities $\mathcal{A} \subset \mathcal{E}$ is available for each query. Formally, given a natural language query q containing a set of keywords $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$, the Open-World KG-RAG is capable of autonomously learning a mapping function $\phi : \mathcal{K} \times \mathcal{G} \rightarrow \hat{\mathcal{A}}$, where $\hat{\mathcal{A}} \subset \mathcal{E}$ is the inferred candidate anchor set. Unlike the entity linking where the target entity is unique, Open-World KG-RAG is proposed for QA tasks where the target entity may not exist or may have multiple candidates.

4 Method

In this section, we propose AnchorRAG, an iterative retrieval RAG pipeline built upon a multi-agent collaborative framework, as shown in the Figure 2. The architectural design is primarily guided by the paradigms of “divide and conquer” and “collaborative refinement”. We avoid the conventional single-threaded and error-prone “link-then-retrieve” linear paradigm. Instead, we realize the complex open-world reasoning task with three specialized sub-tasks, each

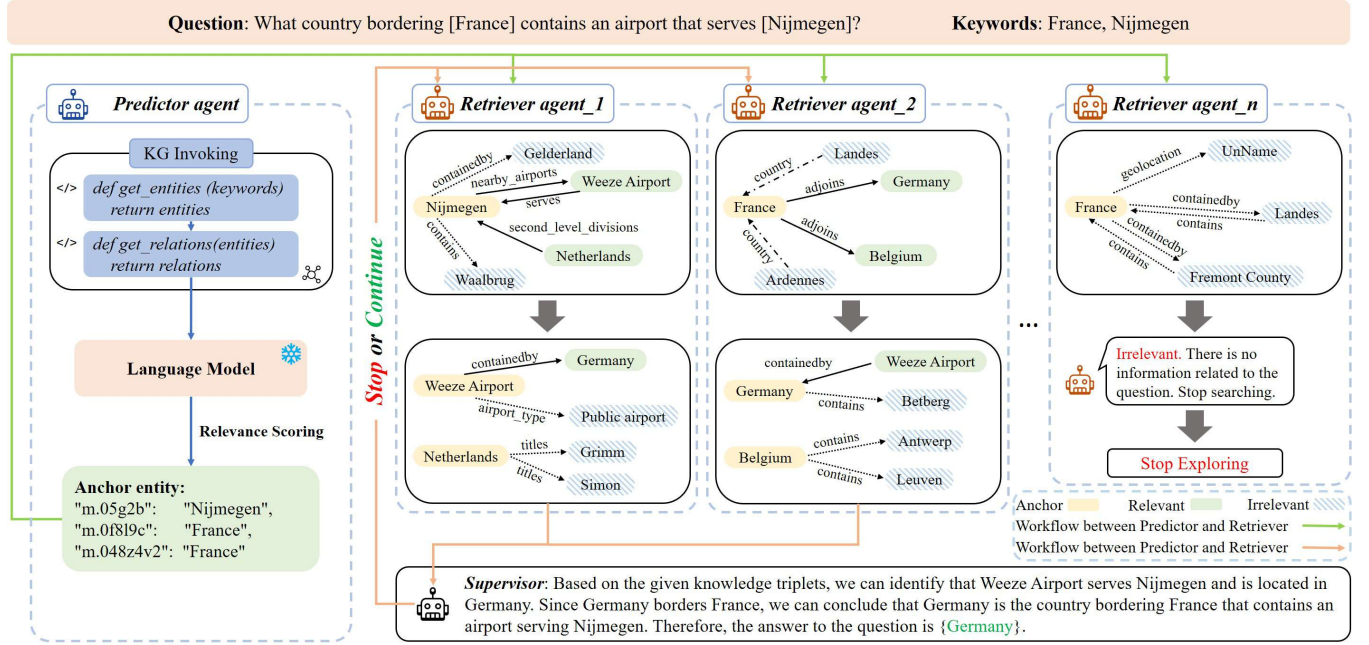


Figure 2: An overview of the AnchorRAG framework, including the collaborative workflows of multi-agent exploration for question answering.

operated by a distinct agent. First, the Predictor Agent is responsible for identifying anchor entities within the user queries and aligning them with the knowledge graph. Then, the Retriever Agents perform multi-hop knowledge graph retrieval and filtering, parallel to cover critical information from multiple perspectives. Finally, the Supervisor Agent supervises the above retrieval processes and generates the answers based on the information retrieved from the given KG. The prompt and in-context examples used in our method can be found in the Appendix E.

4.1 Anchor Entity Identifying

In our framework, the process of accurately identifying anchor entities from natural language questions is handled by the predictor agent. This process consists of the following two steps.

Candidate Entity Recognition. The predictor agent is associated with a LLM to recognize and extract keywords from the input question q , forming a keyword set $\mathcal{W} = \{w_1, w_2, \dots, w_n\}$, where each w_i denotes an entity mention relevant to q . As shown in Figure 2, given the question q : "What country bordering France contains an airport that serves Nijmegen?", the predictor extracts $\mathcal{W} = \{\text{France, Nijmegen}\}$. Considering the issues of entity abbreviations, aliases, long-tail entities, and potential spelling errors in open-world scenarios, directly using $\mathcal{W}$ for exact matching within a large-scale KG [4] often results in low recall and high computational cost. To handle this issue, AnchorRAG employs a robust entity linking mechanism based on exact semantic indexing. Specifically, AnchorRAG encodes all entity names in the KG with MiniLM [36], then conducts a FAISS [7] index from these embeddings, enabling efficient similarity search. On this basis, Predictor first cleans and proofreads the initial set $\mathcal{W}$ of query keywords, and filters out

irrelevant interferences such as non-critical terms, and is prompted to correct potential spelling errors. Subsequently, with the inverted file and product quantization techniques provided by the FAISS library, Predictor can efficiently retrieve a set of the most relevant candidate entities $\tilde{\mathcal{E}}$ from the knowledge graph.

Context-Aware Entity Disambiguation. The initial candidate set $\tilde{\mathcal{E}}$ often contains numerous entities with similar names that are irrelevant to the question's context. Taking all of them into account will increase the overhead of subsequent retrieval and inevitably introduce noise. AnchorRAG is inspired by the assumption that we can effectively determine the entity's relevance by measuring the semantic implied in its local graph structure. An entity is a more probable anchor if its neighboring relations are semantically aligned with the given question. Therefore, the Predictor employs a context-aware entity disambiguation strategy to precisely identify the most relevant candidates. Specifically, for each candidate entity $e \in \tilde{\mathcal{E}}$, the agent gathers its one-hop neighboring relations and obtains the set $\mathcal{R}(e)$. Then it invokes a pre-trained model SBERT [29] to obtain vector representations for the question q and each relation $r_i \in \mathcal{R}(e)$, denoted as $\mathbf{q}$ and $\mathbf{r}_i$ respectively. The relevance score $s(r_i, q)$ between a relation and the question is then calculated using cosine similarity:

$$s(r_i, q) = \frac{\mathbf{r}_i \cdot \mathbf{q}}{\|\mathbf{r}_i\| \|\mathbf{q}\|}, \quad (2)$$

Furthermore, we define the final relevance score for the candidate entity e by aggregating the scores of its most relevant neighboring relations:

$$\text{Score}(e, q) = \frac{1}{k} \sum_{r \in \mathcal{N}(e)} s(r, q), \quad (3)$$

where $\mathcal{N}(e)$ denotes the set of top- k relevant relations according to the Eq.(1). By scoring and ranking all candidates in the set $\tilde{\mathcal{E}}$, the Predictor selects the top- m entities with the higher relevance scores (e.g., France and Nijmegen) as the final anchor set, which will be passed to the following retriever agents for parallel searching.

4.2 Parallel Exploration

With the top- m candidate anchor entities $\{a_1, a_2, \dots, a_m\}$ obtained by the Predictor agent, the framework initiates a parallel exploration operation, assigning each candidate to an independent retriever agent R_i . This multi-agent parallel search is fundamental to the robustness of our framework, as it alleviates the issue of linking to a single, potentially erroneous anchor. Instead of following a linear path, AnchorRAG explores multiple reasoning hypotheses simultaneously. Each retriever agent R_i maintains a reasoning subgraph $G_i^t = (V_i^t, E_i^t)$ at iteration t , initialized as $G_i^{(0)} = (a_i, \emptyset)$. At each iteration, the agent expands its current exploration frontier by retrieving the one-hop neighborhoods of its active nodes, forming a candidate subgraph. To maintain relevance and reduce overhead, this candidate subgraph then undergoes a two-stage refinement process. First, a candidate ranking step scores and selects the most promising relations and entities for expansion. On this basis, an evidence validation step assesses the generated triples, retaining only those that include relevant evidence for the query. The entities within the validated subgraph are considered as the new frontier nodes for the next iteration of expansion.

Candidate Relations Ranking. At the t -th retrieval iteration, each retriever agent dynamically identifies the most relevant relational paths from the active entity e_i^t through semantic alignment. Given the knowledge graph $\mathcal{G}$, the agent first retrieves all one-hop relations connected to e_i^t , denoted as $\mathcal{R}(e_i^t) = \{r_1, r_2, \dots, r_k\}$. Then a LLM acts as a semantic filter that assigns a relevance score $s_r(r_j \mid q, e_i^t)$ to each candidate relation $r_j \in \mathcal{R}(e_i^t)$. The prompt used for this scoring integrates the query context, the current entity, and all retrieved relations, allowing the model to perform context-aware semantic alignment. Subsequently, the Retriever selects the top- b relations with the highest scores, forming a refined relation set $\tilde{\mathcal{R}}_{e_i^t}$. As illustrated in Figure 2, for the given query, Retriever R_1 starts from the anchor entity “Nijmegen” and identifies highly relevant relations such as “nearby_airports”, “serves”, and “second_level_divisions”. Retriever R_2 and Retriever R_3 start from “France”, and then judge that relations like “adjoins” and “containedby” are beneficial to exploring the answer, while filtering out irrelevant relations such as “contains”. During this process, AnchorRAG demonstrates strong generality across diverse knowledge graphs without requiring any training or fine-tuning of models.

Candidate Entities Ranking. At the t -th iteration, for each selected relation $r_j \in \tilde{\mathcal{R}}_{e_i^t}$, the retriever agent R_i traverses $\mathcal{G}$ to obtain all connected entities, forming a candidate entity set $\mathcal{E}(r_j) = \{e_1, e_2, \dots, e_{n_j}\}$. Note that directly evaluating individual entities without considering their relational context may be suboptimal: such approach may inevitably filter out intermediate entities that

are crucial for multi-hop reasoning. To address this problem, R_i employs a LLM-based scoring mechanism that evaluates the contextual relevance $s_e(e_k \mid q, e_i^t, r_j)$ for each entity $e_k \in \mathcal{E}(r_j)$. The structured prompt integrates the query q , the current anchor entity e_i^t , and the relation r_j , allowing the model to perform fine-grained semantic reasoning. All candidate entities are then ranked according to their scores, and for each relation r_j , the top- b entities are retained:

$$\tilde{\mathcal{E}}_{r_j} = \text{Top}_b(\mathcal{E}_{r_j}, s_e) \quad (4)$$

These selected entities will form the preliminary reasoning paths $\mathcal{T}(e_i^t)$, which serve as the evidence for the subsequent validation phase.

In addition, to reduce computational cost, we propose an alternative hybrid retrieval strategy that replaces the LLM-based scoring with a combination of sparse (e.g., BM25) and dense (e.g., SBERT) retrieval models. This hybrid approach allows AnchorRAG to efficiently recall semantically or lexically relevant candidates, balancing efficiency and reasoning performance. The effectiveness of this variant is further evaluated in the efficiency evaluation section.

Evidence Validation. In the candidate ranking stage, each candidate relation r_j and entity e_k is assigned a relevance score $s_r(r_j)$ and $s_e(e_k)$, respectively. Existing methods usually apply a linear combination of these two relevance measures to approximate the similarity of the reasoning path formed by the relation–entity pair. They implicitly assume that the relevance of relations and entities is independent, while neglecting their inherent semantic coherence. As a result, they fail to capture the complex semantic interactions and implicit logical structures within the knowledge graph, and thus limit the retrieval performance. To further locate query-relevant facts hidden in the knowledge graph, AnchorRAG introduces a novel evidence validation mechanism. Specifically, each R_i converts its candidate reasoning paths $\mathcal{T}(e_i^t)$ into natural language text sequences, denoted as $\mathcal{X}_i^t = \{x_1, x_2, \dots, x_{p_i}\}$, including both the query q and contextual information. On this basis, the agent leverages the reasoning and comprehension capabilities of a LLM to identify relevant paths that contain critical evidence, and filter out irrelevant or inconsistent evidence. As illustrated in Figure 2, Retriever R_1 validates the reasoning chain $\mathcal{P}_1 = (\text{Nijmegen, nearby_airports, Weeze Airport, containedby, Germany})$, while Retriever R_2 identifies another path $\mathcal{P}_2 = (\text{France, adjoins, Germany, Weeze Airport, containedby, Germany})$. These validated subgraph $\tilde{G}_i^t$ is then stored in a shared global memory $\mathcal{M}$ accessible to all agents, serving as an important source for subsequent reasoning iterations and answer generation.

4.3 Reasoning

After the parallel exploration phase, each retriever agent R_i has obtained a reasoning subgraph G_i^t (and its validated subset $\tilde{G}_i^t$) which are stored in the shared global memory $\mathcal{M}$. The Supervisor agent $\mathcal{S}$ aggregates these multi-perspective evidences and checks whether the obtained information is sufficient to answer the query q . If the evidence chain is complete and results in a clear conclusion, the Supervisor generates the answer directly. If the evidence is insufficient, Supervisor $\mathcal{S}$ evaluates all candidate reasoning paths

Dataset	WebQSP		GrailQA		CWQ		WebQuestions	
	Hit@1	Acc	Hit@1	Acc	Hit@1	Acc	Hit@1	Acc
Method	Qwen-Plus							
IO	63.3	42.2	33.2	26.6	33.2	33.2	<u>56.3</u>	43.2
Chain-of-Thought	62.9	41.5	32.3	26.7	37.6	37.6	54.1	41.2
Self-Consistency	63.0	41.2	33.8	28.4	39.2	39.2	52.7	40.1
PoG	33.1	25.4	36.9	33.0	39.5	29.2	25.3	21.5
ToG	<u>66.1</u>	<u>46.3</u>	<u>41.9</u>	<u>35.3</u>	<u>39.8</u>	<u>39.8</u>	56.1	<u>43.7</u>
AnchorRAG(Ours)	73.3	56.4	62.7	56.0	47.0	47.0	60.9	50.5
Method	GPT-4o-mini							
IO	65.8	46.1	35.6	27.9	35.7	35.7	57.7	45.4
Chain-of-Thought	62.5	41.6	31.0	25.8	34.5	34.5	53.6	41.2
Self-Consistency	59.2	40.1	34.0	27.1	36.1	36.1	50.7	39.3
PoG	55.3	36.4	39.0	34.2	36.1	36.1	41.8	33.3
ToG	<u>71.4</u>	<u>49.9</u>	<u>48.7</u>	<u>40.9</u>	<u>40.7</u>	<u>40.7</u>	61.4	<u>47.6</u>
AnchorRAG(Ours)	74.1	57.4	63.4	56.8	44.8	44.8	<u>60.0</u>	50.0

Table 1: The results of different methods on various Datasets, using Qwen-Plus and GPT-4o-mini as the LLM. The best results are highlighted in bold. The suboptimal results are underlined.

and drives the high relevance Retriever to explore in a promising direction at the next round, which are marked as [active]. Conversely, retrievers deemed to deviate from the query or retrieve noisy, irrelevant information are terminated early, reducing the overhead and limiting the influence of irrelevant data in the global memory. Exploration stops under either of the following two conditions: (1) a predefined maximum search depth $\mathcal{D}$ is reached, or (2) no retrievers remain in the [active] state. In such cases, the Supervisor concludes that further KG-based path exploration is unlikely to generate an answer and applies the chain-of-thought (CoT)[38] reasoning mechanism instead to directly obtain the answers.

5 Experiments

Dataset and Evaluation. In the experiments, we adopt four benchmark datasets to evaluate the performance of our framework, including three multi-hop Knowledge Graph Question Answering (QA) datasets: WebQuestionSP (WebQSP) [45], GrailQA [10] and Complex WebQuestions (CWQ) [32], and one for Open-Domain Question Answering: WebQuestions [3]. Freebase [4] is utilized as the external Knowledge Graph, which contains the complete knowledge facts that are necessary to answer questions in the above datasets. We select Freebase to ensure fair comparison with established benchmarks[6, 31] whose structural complexity and large scale poses a challenge to evaluating the reasoning capabilities of RAG methods. In addition, we apply the exact match (i.e., Hit@1) and accuracy (i.e., Acc) as evaluation metrics for our method which are widely used in the field of question answering with RAG.

Implementation Settings. We extract all entities from Freebase, encode their name description into vectors using SBERT model (all-MiniLM-L6-v2, without fine-tuning) [29], and subsequently index them in a vector database [7] for downstream entity querying and

matching. The knowledge graph derived from Freebase is deployed on a locally deployed Virtuoso [8] server for our experiments. We selected GPT-4o-mini and Qwen-Plus as the fundamental LLM in our framework. To prevent redundant exploration by agents and reduce computational overhead, we set both the maximum search depth $\mathcal{D}$ and the expansion width b to 3. This configuration aligns with the empirically optimal values identified in [31]. Additionally, we fix the number of neighbor relations k during anchor entity identifying to 5, and the number of retrieval agents m is set to 3 for parallel multi-hop reasoning.

Baselines. Some widely used prompting strategies are selected as the baselines, including vanilla IO prompting [5], chain-of-Thought (CoT) [38], and Self-Consistency [37]. Moreover, we also compare the proposal with two representative RAG methods: Think-on-graph [31] and Plan-on-graph [6]. It is worth noting that the above two methods are both implemented in the closed-world setting where the anchor entities have been pre-defined and are part of the annotated datasets. Thus, to test their performance in the specific scenario where the anchor entity is directly available, we apply the fundamental LLM to select the anchor entities for the following open-world question answering. Other settings and parameters remain unchanged.

5.1 Main Results

The main experimental results, as shown in Table 1, indicate that our proposed method achieves a significant performance advantage over most baseline methods when deployed with both the Qwen-Plus and GPT-4o-mini LLMs. Specifically, when applying Qwen-Plus, AnchorRAG achieves an average improvement by 10% and 11.2% on the metrics Hit@1 and Accuracy respectively, compared to the strongest baseline. Besides, with GPT-4o-mini model,

Dataset	WebQSP		GrailQA	
	normal	open-world	normal	open-world
Method	Qwen-Plus			
ToG	66.1	58.5↓ ^{11.5%}	41.9	32.1↓ ^{23.4%}
AnchorRAG	73.3	71.3↓ ^{2.7%}	62.7	55.4↓ ^{11.6%}
Method	GPT-4o-mini			
ToG	71.4	67.1↓ ^{6.0%}	43.7	35.5↓ ^{18.8%}
AnchorRAG	74.1	70.3↓ ^{5.1%}	63.4	55.4↓ ^{12.6%}

Table 2: Performance comparisons (Hits@1) of different methods on the normal and open-world versions of WebQSP and GrailQA datasets, using Qwen-Plus and GPT-4o-mini as the backbone LLMs.

our proposal also achieves the increases of 6.3% and 8.6%. We can also find that our method obtain an outstanding performance on the complex datasets like GrailQA, where its Hit@1 and Accuracy scores are increased substantially by 20.8% and 20.7%, respectively. The reason may be that the questions and the KG entities are semantically inconsistent in this dataset, and the anchor entities are more difficult to locate in this case. AnchorRAG can handle this issue by the multi-agent collaboration, thereby obtaining the state-of-the-art results. In addition, the performance of the RAG methods, e.g., ToG, consistently surpasses the methods solely relying on the internal knowledge, e.g., CoT. This demonstrates the effectiveness of the RAG methods in the question answering task, especially when dealing with the complex reasoning such as GrailQA and CWQ.

5.2 Robust Analyses

Since existing public QA datasets primarily assume that the external KG contains all the necessary information and rely on clean entity linking, they are ill-suited for the open-world question answering task. Thus, to further evaluate the generality and robustness of our method, we constructed the open-world versions of the WebQSP and GrailQA datasets. Specifically, we first added semantic noise by introducing typos into the original questions. These perturbations can effectively simulate real-world scenarios, thereby testing the capability of the proposed method for the open-world RAG. Then, we incorporated additional questions into the datasets whose answers could not be directly retrieved from the KG. The detailed construction process is provided in the appendix A. The results on the self-constructed datasets are presented in Table 2, where we can observe that our method shows strong robustness on this open-world scenario. For instance, on GrailQA, the performance of ToG using Qwen-Plus drops by 23.4%, whereas our method only drops by 11.6%. These results demonstrate that AnchorRAG can effectively address the open-world QA tasks and further enhance the generality of the RAG methods.

5.3 Ablation Studies

To validate the effectiveness of the core components within our proposed AnchorRAG framework, we conducted the following ablation

Method	WebQSP	GrailQA
AnchorRAG	74.1	63.4
w/o Entity Disambiguation	67.8	42.8
w/o Parallel Exploration	72.8	60.2
w/o LLM Ranking	71.1	64.2
w/o Evidence Validation	71.6	62.8

Table 3: Ablation study of AnchorRAG: Effect of different components on the Hits@1 performance.

studies on the WebQSP and GrailQA datasets, with the results presented in Table 3. Note that “w/o Entity Disambiguation” removes the neighborhood information during anchor entity identification, “w/o Parallel Exploration” eliminates the multi-agent retrieval mechanism, replacing it with a single-path retrieval approach. The “w/o LLM Ranking” variant (referred to as **AnchorRAG-LR** in the efficiency evaluation subsection) uses a lightweight hybrid retrieval strategy instead of LLM ranking. It works by combining SBERT semantic similarity and BM25 scores for efficient candidate filtering. And “w/o Evidence Validation” refers to the absence of fine semantic filtering for candidate triples during retrieval. The results show that each component can contribute to the question answering performance. Comparing with parallel exploration and evidence validation, the entity disambiguation module has a greater impact on the performance, achieving the improvement by 6.3% and 20.6% on WebQSP and GrailQA respectively, which emphasizes the importance of precise anchor entities in the open-world question answering tasks. Interestingly, the “w/o LLM Ranking” variant performs slightly better than the AnchorRAG on GrailQA. This is likely because GrailQA contains more structured and well-aligned relations, where lightweight semantic retrieval (SBERT + BM25) can efficiently the semantic associations without the excessive reasoning conducted by the LLM.

5.4 Efficiency evaluation

In this efficiency evaluation, we compared the performance of AnchorRAG, AnchorRAG-LR along with the strongest baseline ToG in terms of computational efficiency and answer quality. The results are shown in Figure 3, where “Prompt Tokens Consumption (M)” represents the overhead of input tokens that formulate the LLM prompt, “Completion Tokens Consumption (M)” indicates the number of tokens generated in the LLM responses and “Average Inference Time per Sample (s)” measures the average reasoning time for answering each query. From the results, we can obviously observe that AnchorRAG-LR achieves substantially faster reasoning than both ToG and AnchorRAG, while also effectively reducing the overhead of the prompt or completion tokens. For example, on the WebQSP dataset, AnchorRAG-LR reduces the total token consumption by 48.2% compared to ToG and by 77.8% compared to AnchorRAG; its average reasoning time is only 75.3% and 36.2% of that of the methods ToG and AnchorRAG. Similarly, we can observe that on the GrailQA dataset, where token consumption decreases by around 37.4% and 74.2%, and reasoning speed improves by 26.8% and 62.4%, respectively. Additionally, in terms of answer accuracy metrics, AnchorRAG remains superior due to its LLM-based powerful

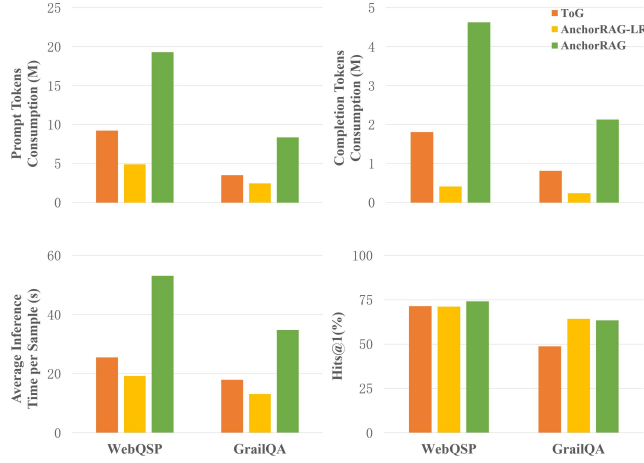


Figure 3: Efficiency and Performance Comparison of Different Methods on the WebQSP and GrailQA Datasets, using GPT-4o-mini as the backbone LLM.

reasoning capability. In summary, AnchorRAG achieves stronger semantic performance at a higher computational cost, whereas AnchorRAG-LR provides a practical and efficient alternative in the resource-constrained scenarios. In addition, we recommend prioritizing the LLM-ranking strategy (AnchorRAG) for those highly ambiguous queries that cannot be easily tackled by lightweight models, e.g., SBERT.

5.5 Effect of the Retrieval

To further validate the effectiveness of our method in anchor identifying and knowledge retrieval, we compare the effect of retrieval of our method against two RAG baselines (PoG and ToG) and a variant (AnchorRAG w/o Parallel Retrieval). The results are shown in Figure 4. The figures in the top row illustrate the accuracy of anchor identifying (marked in green), where our method significantly outperforms the baselines by disambiguating the entities with their textual and neighborhood information. The bottom row presents the proportion of the exact match for the answers, including the answers obtained by retrieval (marked in red) and those inferred by CoT reasoning (marked in orange). Notably, AnchorRAG achieves the highest retrieval rate at 49.6%, indicating that the multi-agent collaborative mechanism can effectively obtain the correct answer entities, which supports the following reasoning process to improve the QA performance.

5.6 Hyper-parameter Analyses

We conducted the parameter sensitivity analyses on GrailQA to investigate the optimal configuration for the number of neighborhood relations k and retrieval agents m , where the results are shown in Table 4. We can find that retaining excessive neighboring relations k for each candidate entity will lead to a performance loss, due to the fact that it may introduce noise and neglect the key structural information. Besides, a proper number of retriever agent m can make AnchorRAG achieve the optimal results, which can reflect the effectiveness of the MAC framework. In addition, we can also find

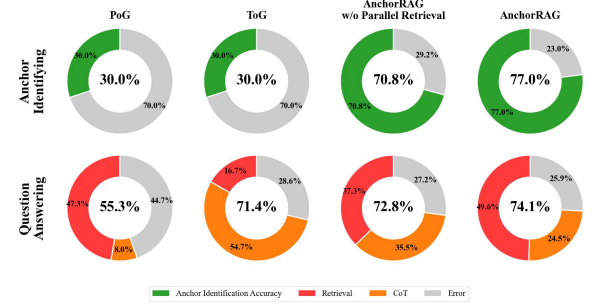


Figure 4: The effect of the anchor identifying and knowledge retrieval on WebQSP.

Setting	$k = 3$	$k = 5$	$k = 7$
$m = 1$	60.4	57.6	56.4
$m = 2$	63.6	63.3	61.0
$m = 3$	63.1	63.4	63.3
$m = 4$	62.6	63.1	62.4

Table 4: The results of AnchorRAG with different hyper-parameters using GPT-4o-mini on GrailQA.

that more retrieval agents won't always obtain the better results. Excessive agents will introduce uncertain retrieval information into the collaborative process, thereby impairing the performance. Furthermore, we also conduct the hyperparameter sensitivity analysis for retrieval depth and width. The detailed experimental results and analyses can be found in the appendix B.

6 Conclusion

In this paper, we present AnchorRAG, a novel multi-agent collaborative RAG framework for open-world Knowledge Graph Question Answering. Unlike existing methods that rely on predefined anchor entities, AnchorRAG overcomes the bottleneck of unreliable entity linking by identifying and grounding plausible anchors without prior assumptions. Specifically, a predictor agent dynamically identifies candidate anchor entities by aligning the given questions with the candidate entities and initializes independent retriever agents to conduct parallel multi-hop explorations. Then a supervisor agent formulates iterative retrieval strategy for these retriever agents and synthesizes the reasoning paths to generate the final answer. Extensive experiments on four public KGQA benchmarks demonstrate that AnchorRAG significantly outperforms the state-of-the-art baselines, validating the effectiveness of our MAC framework, especially when dealing with the complex multi-hop reasoning tasks.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62476274, U22B2048, 62394330), in part by the Special Project of Joint Funding for Municipal, University (Institute) and Enterprise under the Guangzhou Basic Research Program (2024A03J0395), in part by the Science and Technology Projects in Guangzhou (2024D03J0010).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. Dbpedia: A nucleus for a web of open data. In *international semantic web conference*. Springer, 722–735.
- [3] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on freebase from question-answer pairs. In *Proceedings of the 2013 conference on empirical methods in natural language processing*. 1533–1544.
- [4] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: a collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 1247–1250.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [6] Liyi Chen, Panrong Tong, Zhongming Jin, Ying Sun, Jieping Ye, and Hui Xiong. 2024. Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs. *Advances in Neural Information Processing Systems* 37 (2024), 37665–37691.
- [7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [8] Orri Erling and Ivan Mikhailov. 2009. RDF Support in the Virtuoso DBMS. In *Networked Knowledge-Networked Media: Integrating Knowledge Management, New Media Technologies and Semantic Systems*. Springer, 7–24.
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [10] Yu Gu, Sue Kase, Michelle Vanni, Brian Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond iid: three levels of generalization for question answering on knowledge bases. In *Proceedings of the web conference 2021*. 3477–3488.
- [11] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv:2405.14831* [cs.CL] <https://arxiv.org/abs/2405.14831>
- [12] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. 2025. Retrieval-Augmented Generation with Graphs (GraphRAG). *arXiv:2501.00309* [cs.IR] <https://arxiv.org/abs/2501.00309>
- [13] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [15] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2025. GRAG: Graph Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 4145–4157. doi:10.18653/v1/2025.findings-naacl.232
- [16] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* (2025), 1–55.
- [17] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403* (2024).
- [18] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Structgpt: A general framework for large language model to reason over structured data. *arXiv preprint arXiv:2305.09645* (2023).
- [19] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551* (2017).
- [20] Jiho Kim, Yeonsu Kwon, Yohan Jo, and Edward Choi. 2023. KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models. *arXiv:2310.11220* [cs.CL] <https://arxiv.org/abs/2310.11220>
- [21] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Complex knowledge base question answering: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 11 (2022), 11196–11215.
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [23] Belinda Z. Li, Sewon Min, Srinivasan Iyer, Yashar Mehdad, and Wen-tau Yih. 2020. Efficient One-Pass End-to-End Entity Linking for Questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6433–6441. doi:10.18653/v1/2020.emnlp-main.522
- [24] Yangning Li, Weizhi Zhang, Yuyao Yang, Wei-Chieh Huang, Yaozu Wu, Junyu Luo, Yuanchen Bei, Henry Peng Zou, Xiao Luo, Yusheng Zhao, et al. 2025. Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs. *arXiv preprint arXiv:2507.09477* (2025).
- [25] Zhenyu Li, Sunqi Fan, Yu Gu, Xiuxing Li, Zhichao Duan, Bowen Dong, Ning Liu, and Jianyong Wang. 2024. Flexkbqa: A flexible llm-powered framework for few-shot knowledge base question answering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 18608–18616.
- [26] Haoran Luo, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, et al. 2023. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. *arXiv preprint arXiv:2310.08975* (2023).
- [27] Jie Ma, Zhitao Gao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, et al. 2025. Debate on graph: a flexible and reliable reasoning framework for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 24768–24776.
- [28] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. *arXiv:2408.08921* [cs.AI] <https://arxiv.org/abs/2408.08921>
- [29] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).
- [30] Wei Shen, Jianyong Wang, and Jiawei Han. 2015. Entity Linking with a Knowledge Base: Issues, Techniques, and Solutions. *IEEE Transactions on Knowledge and Data Engineering* 27, 2 (2015), 443–460. doi:10.1109/TKDE.2014.2327028
- [31] Jiahuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697* (2023).
- [32] Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643* (2018).
- [33] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 3505–3522. doi:10.1145/3696410.3714892
- [34] Daniel Vollmers, Hamada Zahera, Diego Moussallem, and Axel-Cyrille Ngomo. 2025. Contextual Augmentation for Entity Linking using Large Language Models. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 8535–8545. <https://aclanthology.org/2025.coling-main.570/>
- [35] Song Wang, Junhong Lin, Xiaojie Guo, Julian Shun, Jundong Li, and Yada Zhu. 2025. Reasoning of large language models over knowledge graphs with super-relations. *arXiv preprint arXiv:2503.22166* (2025).
- [36] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MINLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 485, 13 pages.
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [39] Yuan Xia, Jingbo Zhou, Zhenhui Shi, Jun Chen, and Haifeng Huang. 2025. Improving retrieval augmented language model with self-reasoning. In *Proceedings of the AAAI conference on artificial intelligence*. 25534–25542.
- [40] Derong Xu, Xinhang Li, Ziheng Zhang, Zhenxi Lin, Zhihong Zhu, Zhi Zheng, Xian Wu, Xiangyu Zhao, Tong Xu, and Enhong Chen. 2025. Harnessing large language models for knowledge graph question answering via adaptive multi-aspect retrieval-augmentation. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'25/IAAI'25/EAAI'25)*. AAAI Press, Article 2849, 9 pages. doi:10.1609/aaai.v39i24.34747
- [41] Yao Xu, Shizhuo He, Jiabei Chen, Zihao Wang, Yangqiu Song, Hanghang Tong, Guang Liu, Kang Liu, and Jun Zhang. 2024. Generate-on-graph: Treat llm as both

agent and kg in incomplete knowledge graph question answering. *arXiv preprint arXiv:2404.14741* (2024).

- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [43] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [44] Xi Ye, Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, and Caiming Xiong. 2022. RNG-KBQA: Generation Augmented Iterative Ranking for Knowledge Base Question Answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 6032–6043. doi:10.18653/v1/2022.acl-long.417
- [45] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 201–206.
- [46] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958* (2025).
- [47] Qinggang Zhang, Junnan Dong, Hao Chen, Daochen Zha, Zailiang Yu, and Xiao Huang. 2024. Knowgpt: Knowledge graph based prompting for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 6052–6080.

A Details of Experimental Datasets

We use four benchmark datasets: WebQuestionSP (WebQSP) [45], GrailQA [10] and Complex WebQuestions (CWQ) [32], and WebQuestions (WebQ) [3] to evaluate the effectiveness of our method. These datasets include a training set, a validation set, and a test set. For fair comparison, we follow the baselines [6, 31] and use only the test sets as evaluation benchmarks. In order to save computational resources, following prior research [6, 31], we only used a sample of 1000 data points from the GrailQA dataset for evaluation. Noticing that, in order to verify the performance of each method in open world settings, we introduce noise into the WebQSP and GrailQA datasets, rewriting keywords in the questions to make it impossible to directly match entities in the knowledge graph through string matching. We also sample a small portion of questions from two other datasets, HotpotQA [43] and TriviaQA [19], and merge them

Datasets	#Train	#Validation	#Test
WebQSP	3098	-	1639
GrailQA	44,337	6763	13231
CWQ	27734	3480	3475
Webquestions	3778	-	2032
WebQSP(Open-World)	3098	-	1803
GrailQA(Open-World)	44,337	6763	1100

Table 5: Statistics of datasets.

Datasets	1 hop	2 hop	≥ 3 hop
WebQSP	65.49%	34.51%	0.00%
GrailQA	68.92%	25.82%	5.26%
CWQ	40.91%	38.34%	20.75%

Table 6: statistics of the reasoning hops in WebQSP, CWQ and GrailQA.

with the noisy data from WebQSP and GrailQA to create two open world datasets (Open-World WebQSP and Open-World GrailQA) to verify the robustness of our method. The statistics of the datasets are shown in Table 5, and the statistics of the reasoning hops are shown in Table 6.

B Experiments of Additional Hyper-parameter

Figure 5 shows the hyperparameter sensitivity analysis for retrieval depth and width. The experimental results strongly validate the effectiveness of our default configuration, i.e., width=3 and depth=3. On both datasets, the Hit@1 performance of our method peaks when these settings are used. This non-linear performance trend is attributed to two reasons. First, the performance improvement is limited by the inherent complexity of the datasets, as most questions do not require more than 3 hops of reasoning, as shown in table 6. Therefore, a deeper search does not yield significant benefits. Second, excessively expanding the search space introduces a significant amount of noise from irrelevant entities and relations, which have the negative impact on the reasoning and lead to a poorer performance. This is shown by the performance degradation on GrailQA when depth or width is increased to 4. In particular, the WebQSP dataset does not contain questions requiring more than 3 hops. Consequently, increasing the retrieval depth beyond 3 on WebQSP offers no performance benefit, and we therefore did not perform experiments with depth=4 on this dataset. To sum up, our parameter selection presents an optimal trade-off between maintaining adequate search space, mitigating noise, and balancing computational overhead.

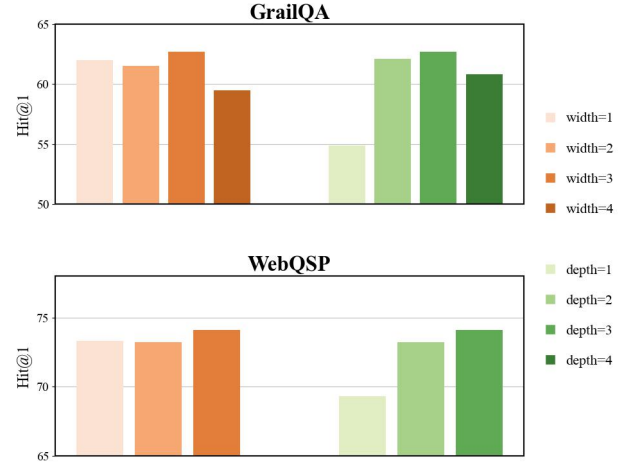


Figure 5: Performances of AnchorRAG with different search depths and widths.

C Experiments on Additional LLM Backbones

To further evaluate the universality and robustness of AnchorRAG with different Large Language Model (LLM) backbones, we extend our experiments using two additional state-of-the-art models:

Dataset	GrailQA		
	Hit@1	Acc	F1
Claude4.5-haiku			
ToG	31.7	26.6	27.6
AnchorRAG	45.4	39.0	40.3
Gemini-2.5-flash			
ToG	39.6	32.8	34.0
AnchorRAG	56.1	47.9	49.7

Table 7: Performance comparison of ToG and AnchorRAG using additional LLM backbones (Claude4.5-haiku and Gemini-2.5-flash) on GrailQA datasets.

Claude4.5-haiku and **Gemini-2.5-flash**. We compared the performance of our method against the strongest baseline, i.e., ToG, on the GrailQA dataset. The experimental results are presented in Table 7. Consistent with the results in the main experiments, AnchorRAG demonstrates superior performance over the ToG on both new backbones. For example, on the GrailQA dataset, AnchorRAG achieves substantial improvements in Hit@1, surpassing ToG by **13.7%** with Claude4.5-haiku and **16.5%** with Gemini-2.5-flash. This significant improvement validates that our multi-agent collaboration framework effectively mitigates the entity linking issues in complex open-world scenarios, regardless of the underlying LLM. These findings suggest that AnchorRAG’s effectiveness is model-agnostic and can cooperate with various LLMs to enhance open-world reasoning capabilities.

D Case Study

To intuitively demonstrate the superiority of our method in knowledge graph retrieval, we conduct a case study that compares our approach with the ToG baseline on a representative complex question from the CWQ dataset: “Who inspired F. Scott Fitzgerald, and who was the architect that designed The Mount?”. As shown in Figure 6, we can observe that ToG can’t locate both the anchor

entities, resulting in meaningless reasoning paths and generating an incorrect answer. In contrast, our multi-agent framework can effectively identify the anchor entities for the given questions by the entity grounding mechanism. With assigning multi retriever agents to independently explore different reasoning paths, AnchorRAG accurately locates the correct answer entity “Edith Wharton”. Additionally, this case highlights the robustness of our method. For example, although it initially considers an incorrect entity (“The Weapon”) as an anchor entity, it promptly detects the irrelevance of the following retrieval path with the rough pruning and fine filtering method, and terminates the corresponding agent, which prevents unnecessary computation overhead.

Question: Who inspired Scott Fitzgerald that is the architect who designed The Mount?

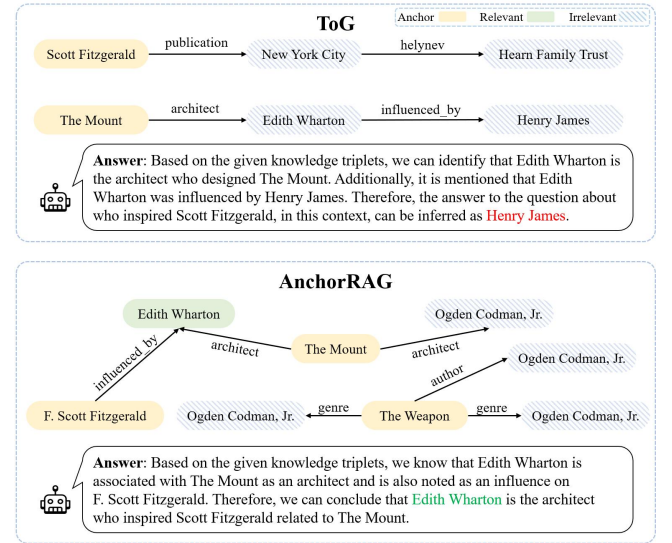


Figure 6: Comparisons of retrieval process between a baseline reasoning (ToG) and our method (AnchorRAG) on a complex question. The entities marked in yellow are anchor entities, marked in green are correct answer entities. The ones marked in blue and striped are invalid entities.

E Prompts

The prompts used in our method are detailed in this section.

Prompt for Candidate Entity Generation

Extract all topic entities from the given multi-hop question. Topic entities are proper nouns, named entities, or specific concepts that are crucial for retrieving external knowledge. They may come from different sub-questions that contribute to the final answer. If there are multiple topic entities, separate them with commas.

In-Context Few-shot

Question: <Question>

Prompt for Relations Pruning

Please retrieve %s relations (separated by semicolon) that contribute to the question and rate their contribution on a scale from 0 to 1 (the sum of the scores of %s relations is 1).

In-Context Few-shot

Question: <Question>

Topic Entity: <Topic Entity>

Relations: <Relations>

Prompt for Entities Pruning

Please score the entities' contribution to the question on a scale from 0 to 1 (the sum of the scores of all entities is 1).

In-Context Few-shot

Question: <Question>

Relation: <Relation>

Entities: <Entities>

Prompt for Triples Filtering

Given the question and a list of knowledge triples, identify and return only the triples that are directly relevant to answering the question. Do not change the format of the triples. Do not generate any explanations or extra text. Return only the filtered triples as-is.

In-Context Few-shot

Question: <Question>

Triples: <Triples>

Prompt for Triples Evaluating

Given a question and the associated retrieved knowledge graph triplets (entity, relation, entity), you are asked to answer whether it's sufficient for you to answer the question with these triplets and your knowledge (Yes or No).

In-Context Few-shot

Question: <Question>

Triples: <Triples>